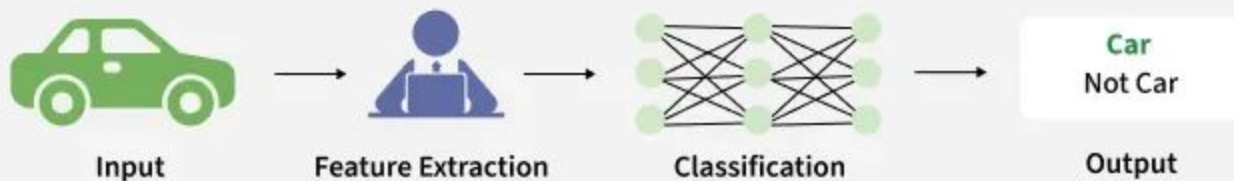


# **Convolutional Neural Networks**

## **(CNNs)**

## Machine Learning



Uses algorithms and learns on its own but may need human intervention to correct errors

---

## Deep Learning



Uses advanced computing, its own neural network, to adapt with little to no human intervention

# Traditional Neural Networks (ANNs) - The Basics

## Architecture:

- Input Layer: Receives raw data.
- Hidden Layers: One or more layers where computations occur.
- Output Layer: Produces the final prediction/classification.
- **Fully Connected:** Every neuron in one layer connects to every neuron in the next.

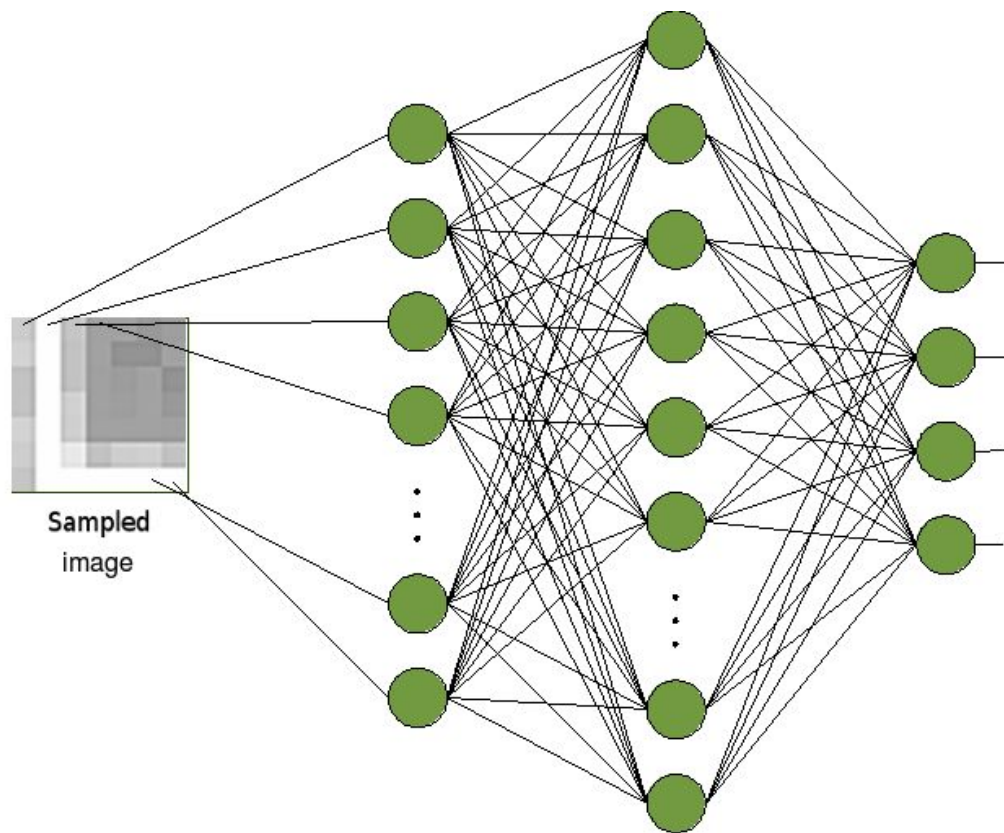
## Data Processing:

- Inputs are typically flattened into a 1D vector.
- This can lead to a loss of spatial information for images.

## Limitations for Images:

- Very large number of parameters for image data (e.g., a 100x100 image flattened is 10,000 inputs).
- Computationally expensive.
- Prone to overfitting due to high parameter count.
- Not inherently translation invariant (requires learning features at every possible position).

# Using ANN for image recognition



Pixel 700\*700  
How many Input  
Features?

# Convolutional Neural Networks (CNNs) - The Innovation

**Key Idea:** Leverage the spatial structure of data.

**Inspired by:** The human visual cortex (hierarchical processing, local receptive fields).

## **Specialized Layers:**

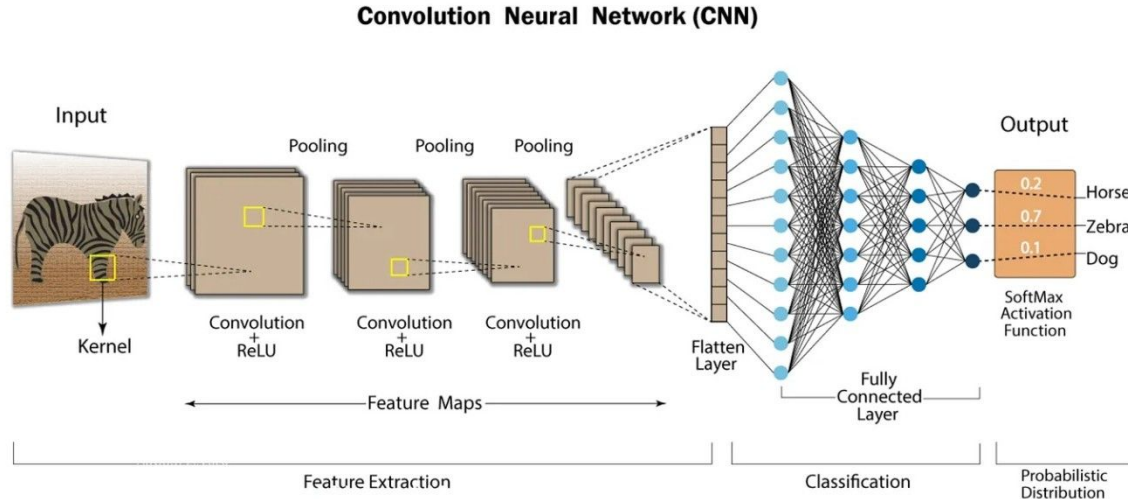
- **Convolutional Layers:**
  - Apply "filters" (small matrices of weights) that slide across the input.
  - Extract local features (edges, textures, patterns).
  - **Local Connectivity:** Neurons only connect to a small region of the previous layer.
  - **Parameter Sharing:** The same filter is used across the entire input, significantly reducing parameters.
- **Pooling Layers:**
  - Reduce the spatial dimensions of the feature maps (down-sampling).
  - Retain the most important information (e.g., max-pooling).
  - Contribute to translation invariance.
- **Fully Connected Layers:**
  - Typically at the end of the CNN.
  - Take the high-level features extracted by convolutional/pooling layers and perform final classification or regression.

# Convolutional Neural Networks

- Is a combination of Deep Neural Network

AND

- Convolutional Kernel with a NN



# What is a CNN?

CNN is a neural network that uses a specialized operation called **convolution** in at least one of its layers.

This operation allows the network to automatically learn features from the input data, unlike traditional Artificial Neural Networks (ANNs) where feature engineering might be manual or less efficient for spatial data.

The foundational concepts behind Convolutional Neural Networks (CNNs) were developed over several decades, with key contributions from various researchers.

**Yann LeCun** is widely credited for establishing and popularizing modern CNNs, particularly with his work in the late 1980s and 1990s.

Here's a brief timeline of important milestones:

- **1980s - Kunihiro Fukushima's - Neocognitron:** Inspired by Hubel and Wiesel's research on the visual cortex, Fukushima developed the **Neocognitron**. This model introduced concepts like hierarchical processing, local connectivity, and a form of pooling, laying significant groundwork for CNNs.
- **Late 1980s - Yann LeCun and colleagues:** Building upon the ideas of the Neocognitron, Yann LeCun, together with researchers like Leon Bottou, Patrick Haffner, and others, made a crucial leap. In **1989**, LeCun published a paper demonstrating how **backpropagation** (the method for training neural networks) could be effectively applied to train CNNs. This allowed CNNs to learn their filters from data, rather than having them hand-designed.
- **1998 - Yann LeCun and colleagues - LeNet-5:** LeCun and his team further refined their work and introduced **LeNet-5**. This was a pioneering 7-layer CNN architecture specifically designed for handwritten digit recognition (like those used for reading zip codes on envelopes). LeNet-5 showcased the practical effectiveness of CNNs and cemented many of the architectural principles still used today (convolutional layers, pooling layers, and fully connected layers).

While the underlying ideas have roots in earlier biological and computational models, it was **Yann LeCun's** work on efficiently training these networks with backpropagation and demonstrating their power with models like LeNet-5 that truly established CNNs as a viable and powerful approach, especially for image recognition tasks.



# Hands on example

```
I = [[ 1,  2,  3,  4,  5,  6,  7,  8],  
      [ 9, 10, 11, 12, 13, 14, 15, 16],  
      [17, 18, 19, 20, 21, 22, 23, 24],  
      [25, 26, 27, 28, 29, 30, 31, 32],  
      [33, 34, 35, 36, 37, 38, 39, 40],  
      [41, 42, 43, 44, 45, 46, 47, 48],  
      [49, 50, 51, 52, 53, 54, 55, 56],  
      [57, 58, 59, 60, 61, 62, 63, 64]]
```

```
K = [[1, 0, -1],  
      [1, 0, -1],  
      [1, 0, -1]]
```

```
[[ 1,  2,  3],  
 [ 9, 10, 11],  
 [17, 18, 19]]
```

```
1*1+2*0+3*-1  
+9*1+10*0+11*-1  
+17*1+18*0+19*-1  
=1+0+-3  
+9+0+-11  
+17+0+-19  
=-2-2-2 = -6
```

```
[[1, 0, -1],  
 [1, 0, -1],  
 [1, 0, -1]]
```

```
[[ 2,  3,  4],  
 [10, 11, 12],  
 [18, 19, 20]]
```

```
[[1, 0, -1],  
 [1, 0, -1],  
 [1, 0, -1]]
```

```
2*1+3*0+4*-1  
+10*1+11*0+12*-1  
+18*1+19*0+20*-1  
=-2  
-2  
-2  
=-2-2-2 = -6
```

## Code -

Color - showConv1.py

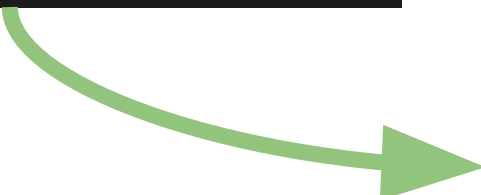
BW - showConv.py

<https://colab.research.google.com/drive/1KFbg6fltryFQOwh3-QD-SMYJi4I3Irkn?usp=sharing>

# Convolution

```
[ [ 50  50  50  50 200 200 200 200]
  [ 50  50  50  50 200 200 200 200]
  [ 50  50  50  50 200 200 200 200]
  [ 50  50  50  50 200 200 200 200]
  [100 100 100 100 150 150 150 150]
  [100 100 100 100 150 150 150 150]
  [100 100 100 100 150 150 150 150]
  [100 100 100 100 150 150 150 150]]
```

```
[[0.11111111 0.11111111 0.11111111]
 [0.11111111 0.11111111 0.11111111]
 [0.11111111 0.11111111 0.11111111]]
```



```
[ [ 22  33  33  66 100 133 133  88]
  [ 33  50  50 100 150 200 200 133]
  [ 33  50  50 100 150 200 200 133]
  [ 44  66  66 105 144 183 183 122]
  [ 55  83  83 111 138 166 166 111]
  [ 66 100 100 116 133 149 149  99]
  [ 66 100 100 116 133 149 149  99]
  [ 44  66  66  77  88  99  99  66]]
```

# Key Components of a CNN

## Convolutional Layer:

- **Filters (Kernels):** This is the heart of a CNN. A convolutional layer uses multiple small matrices called "filters" or "kernels."
- **Feature Detection:** Each filter is designed (or rather, *learned* by the network during training) to detect a specific type of feature in the input, such as edges, corners, textures, or more complex patterns.
- **Sliding Window:** The filter slides across the input image (or the output of a previous layer), performing element-wise multiplication with the underlying pixels and summing the results.
- **Feature Maps:** The result of applying a filter across the entire image is a "feature map," which highlights where that specific feature is present in the input. A convolutional layer will typically have many different filters, producing many different feature maps.
- **Parameter Sharing:** A critical aspect is that the same filter (set of weights) is used across the entire image. This means if a filter learns to detect a vertical edge in one part of the image, it can detect that same vertical edge anywhere else in the image without needing a new set of weights. This significantly reduces the number of parameters compared to ANNs and helps with translation invariance.

# Key Components of a CNN

## Activation Function (e.g., ReLU):

- After the convolution operation, an activation function (like ReLU - Rectified Linear Unit) is applied element-wise to the feature map. This introduces non-linearity, allowing the network to learn more complex relationships.

## Pooling Layer (e.g., Max Pooling):

- Pooling layers reduce the spatial dimensions (width and height) of the feature maps.
- Common pooling operations include Max Pooling (taking the maximum value in a small window) or Average Pooling.
- **Benefit:** This down-sampling helps to reduce computational cost, control overfitting, and provide a degree of "translation invariance" – meaning small shifts in the input image won't drastically change the feature map, making the network more robust.

## Fully Connected Layers:

- After several convolutional and pooling layers (which extract hierarchical features), the learned features are "flattened" into a 1D vector and fed into one or more standard fully connected layers, similar to those in a traditional ANN.
- These layers perform the final classification or regression based on the high-level features extracted by the preceding convolutional layers.

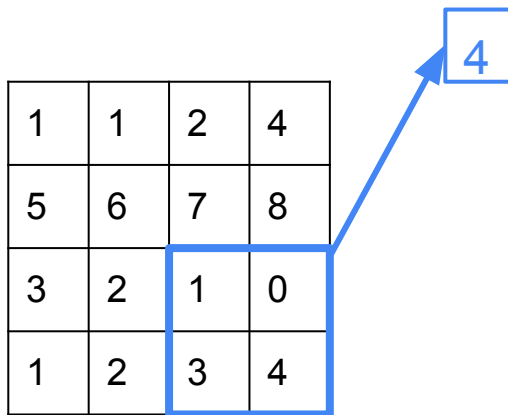
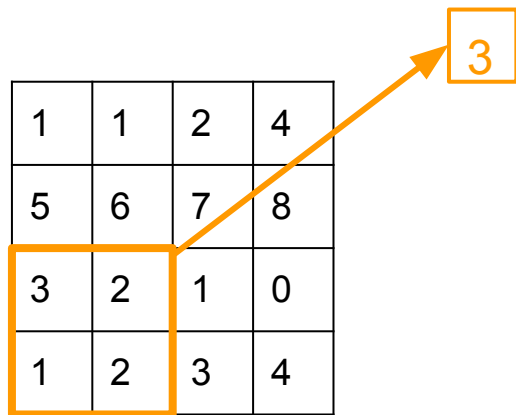
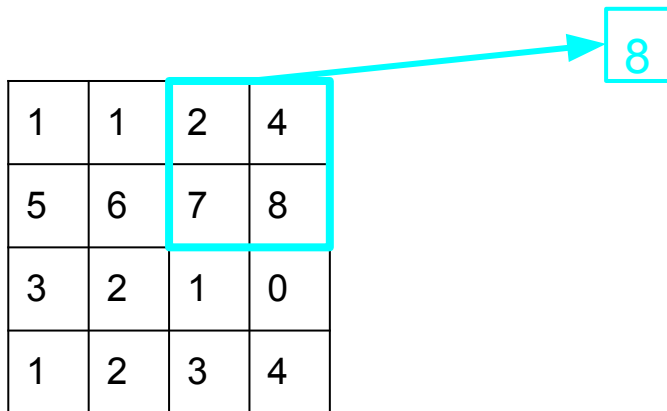
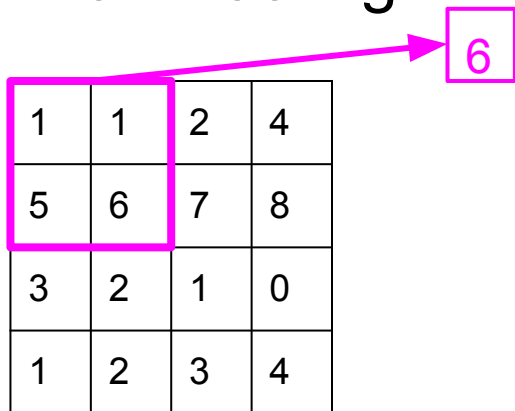
# Pooling layer

A **Pooling Layer** is another crucial component in Convolutional Neural Networks (CNNs) that typically comes after a Conv2D layer and its activation function (like ReLU). Its primary role is to **reduce the spatial dimensions (width and height)** of the feature maps, which serves several important purposes.

## What Does a Pooling Layer Do?

1. **Dimensionality Reduction:** It significantly reduces the number of parameters and computation in the network. This makes the network faster and helps to control overfitting.
2. **Feature Abstraction:** It summarizes the presence of features in regions of the input. Instead of needing the exact location of a feature, it tells you if a feature is present *within a certain area*.
3. **Translation Invariance:** This is a key benefit. By taking the maximum (or average) over a small region, the exact position of a feature within that region becomes less important. If a feature (like an edge) shifts slightly within its receptive field, the max pooling operation will likely still pick it up, making the network more robust to minor shifts or distortions in the input image.

# Max Pooling



|   |   |
|---|---|
| 6 | 8 |
| 3 | 4 |

# CNN and NN

## **Artificial Neural Networks (ANNs):**

- Inspired by the human brain.
- Composed of interconnected "neurons" organized in layers.
- Learn patterns and relationships in data.
- General-purpose, used for various tasks (classification, regression).

## **Convolutional Neural Networks (CNNs):**

- A specialized type of ANN.
- Specifically designed for processing grid-like data (e.g., images).
- Revolutionized computer vision.



# Key Differences

| Feature                | Traditional Neural Networks (ANNs)                       | Convolutional Neural Networks (CNNs)              |
|------------------------|----------------------------------------------------------|---------------------------------------------------|
| Primary Use            | General-purpose (tabular data, text)                     | Image, video, and grid-like data processing       |
| Architecture           | Fully Connected Layers                                   | Convolutional, Pooling, Fully Connected Layers    |
| Connectivity           | Every neuron connected to all in next layer              | Local connectivity in conv. layers                |
| Parameter Sharing      | No (each connection has unique weight)                   | Yes (filters shared across input)                 |
| Feature Extraction     | Manual or implicit learning (less effective for spatial) | Automatic, hierarchical feature learning          |
| Spatial Awareness      | Low (flattens input)                                     | High (preserves spatial structure)                |
| Translation Invariance | Low                                                      | High (recognizes features regardless of position) |
| Parameter Count        | High for image data (prone to overfitting)               | Much lower (more efficient)                       |

# Why ANN dont work well in image recognition

- Too much computation
- Treats neighboring pixels and far away pixels same
- Location of an object in a image matter a lot

## Why CNNs Excel for Images

- **Automatic Feature Learning:** No need for manual feature engineering.
- **Reduced Complexity:** Fewer parameters lead to faster training and better generalization.
- **Spatial Hierarchy:** Naturally learn features from simple to complex.
- **Translation Invariance:** Robust to variations in object position.

# Applications

## **ANNs:**

- Spam detection
- Financial forecasting
- Basic pattern recognition
- Regression problems

## **CNNs:**

- Image Recognition (e.g., facial recognition, self-driving cars)
- Object Detection
- Medical Image Analysis
- Video analysis
- Image generation
- Natural Language Processing (certain tasks)

# Convolutional Neural Networks

**Autonomous feature extraction:** CNNs can automatically learn and extract relevant features from raw data without requiring manual feature engineering, unlike traditional algorithms such as SVMs and decision trees that rely on pre-defined or hand-crafted features.

**Large-scale processing capability:** CNNs excel at handling high-dimensional data and can process features at scale, making them particularly effective for complex tasks like image recognition where thousands of features need to be identified and processed simultaneously.

**Enhanced efficiency through automation:** By eliminating the need for domain experts to manually design and select features, CNNs streamline the machine learning pipeline and reduce the time and expertise required for model development compared to classic approaches.

**Hierarchical feature learning:** CNNs build features in layers, starting with simple patterns and progressively combining them into more complex representations, whereas traditional algorithms like SVMs and decision trees work with a fixed set of input features throughout the learning process.

# Convolutional Neural Networks

**Translation invariance:** Convolutional layers enable CNNs to **recognize patterns** and **objects regardless** of their **position within the input data**, meaning a feature detected in one location can be identified when it appears elsewhere in the image or dataset.

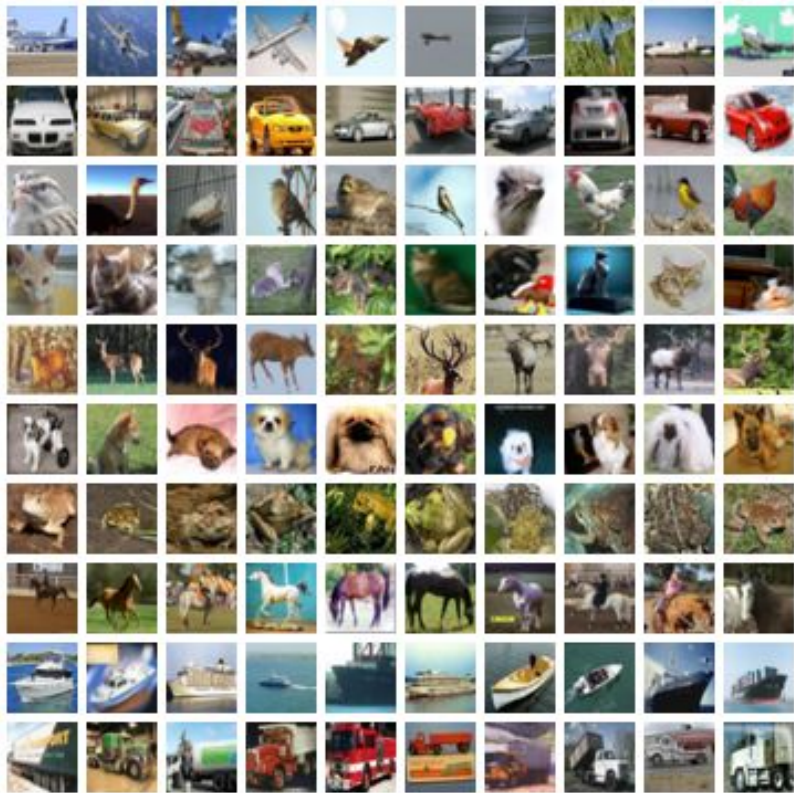
**Position-independent feature detection:** The sliding window approach of convolutional filters allows CNNs to extract the same features whether they appear in the top-left corner, center, or any other location within the input space.

**Orientation robustness:** CNNs can identify patterns and features even when they are rotated or oriented differently from their original training examples, making them adaptable to real-world variations in data presentation.

**Scale flexibility:** The hierarchical structure of convolutional layers enables CNNs to detect features at multiple scales, from fine-grained details to larger structural patterns, regardless of the overall size of the input.

**Enhanced pattern recognition:** By maintaining these invariant properties, CNNs can generalize better to new, unseen data that may contain the same underlying patterns presented in different spatial configurations or transformations.

# CIFAR-10 dataset



<https://www.kaggle.com/c/cifar-10/>

CIFAR-10 is an established computer-vision dataset used for object recognition. It is a subset of the 80 million tiny images dataset and consists of 60,000 32x32 color images containing one of 10 object classes, with 6000 images per class. It was collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton.

# CNN implementation

## Loads and Preprocesses Data:

- Loads the CIFAR-10 dataset,
- Normalizes the pixel values to a 0-1 range (By dividing by 255) ,
- Defines the class names.

Defines CNN Architecture: It constructs a sequential CNN model with:

1. Multiple Conv2D layers for feature extraction, each followed by a relu activation function.
2. MaxPooling2D layers to downsample the feature maps, reducing dimensionality and making the model more robust to minor shifts.
3. A Flatten layer to convert the 2D feature maps into a 1D vector.
4. Dense (fully connected) layers for classification, with a final softmax activation to output probabilities for each of the 10 classes.

1 -> Conv2D layer applies a set of learnable "filters" (or kernels) across the input image (or feature maps from a previous layer). Each filter performs a convolution operation, effectively detecting a specific pattern or feature.

```
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
```

### Why Multiple Layers

- The first Conv2D layers in a CNN tend to learn very **simple, low-level features**. Imagine filters that learn to detect like edges(Horizontal, vertical etc) and texture etc
- **Deeper Layers (High-Level Features)**: We stack more Conv2D layers, the network learns to combine these simpler, low-level features into increasingly **complex and abstract features**.

2-> **ReLU Activation Function. What it does:** ReLU stands for Rectified Linear Unit. Mathematically:

- If the input value is positive, it outputs the input value.
- If the input value is zero or negative, it outputs zero.
- $f(x) = \max(0, x)$



# The combination of multiple Conv2D layers and ReLU activation functions works like this:

1. **Conv2D**: Acts as a "feature detector," scanning the image for specific patterns.
2. **ReLU**: Acts as a "filter/threshold," allowing strong, positive feature detections to pass through and effectively "turning off" weak or irrelevant activations, introducing the necessary non-linearity for deep learning.

This iterative process across multiple layers allows the CNN to build

- hierarchical understanding of the visual content,
- starting from simple components and progressing to complex,
- abstract representations necessary for accurate classification.

# CNN implementation

**Compile the Model: Configure the model for training using the Adam optimizer and `sparse_categorical_crossentropy` as the loss function (suitable for integer labels).**

Compile a deep learning model in Keras (like the CNN for CIFAR-10) is essentially configuring it for the training process.

Next, to tell the model *how* it should learn. This involves specifying three crucial things:

1. **Optimizer:** How the model's weights will be updated during training to minimize the loss.
2. **Loss Function:** How the model measures its own performance (how "wrong" its predictions are).
3. **Metrics:** What quantities to monitor during training and evaluation (e.g., accuracy).

# The Adam Optimizer

## What is an Optimizer?

The goal of training is to adjust the model's internal parameters (weights and biases) so that it can make accurate predictions. This adjustment is done iteratively. An **optimizer** is an algorithm that determines *how* to change these weights and biases based on the loss calculated after each training step.

**How Adam Works (Simplified): Adam** (short for **Adaptive Moment Estimation**) is one of the most popular and effective optimization algorithms. It's an extension of the basic Stochastic Gradient Descent (SGD) algorithm, but with several enhancements:

- **Adaptive Learning Rates:** Unlike simpler optimizers that use a single learning rate for all parameters, Adam calculates *individual learning rates* for different parameters. This means some weights might be adjusted by larger steps, while others by smaller, more precise steps. This adaptability helps in faster and more stable convergence.
- **Momentum:** Adam incorporates the concept of "momentum," which helps the optimizer continue moving in a consistent direction, even when encountering small bumps or plateaus in the loss landscape. It's like building up speed downhill, helping to overcome local minima.
- **Adaptive Beta Parameters:** It uses exponentially decaying averages of past gradients and squared gradients, which helps in efficient updates.

## Why Adam is Used:

- **Fast Convergence:** It often converges faster than other optimizers.
- **Good Performance:** Tends to perform well across a wide variety of tasks and network architectures.
- **Less Hyperparameter Tuning:** It often requires less manual tuning of the learning rate compared to other optimizers.

# Loss Function - The `sparse_categorical_crossentropy`

**What is a Loss Function?** A **loss function** (or cost function) quantifies how "bad" your model's prediction is compared to the actual true value. The optimizer's job is to minimize this loss. A lower loss value means the model is performing better.

## `sparse_categorical_crossentropy`:

Is commonly used for **multi-class classification problems**, where each input sample belongs to *exactly one* class (eg. a cat, or a dog, or a bird, but not multiple at once).

The "sparse" part is key here and relates directly to the format of your labels:

- **`sparse_categorical_crossentropy`**: Used when your true labels are represented as **integers**.
  - **Example (CIFAR-10)**: In CIFAR-10, the labels are single integers like `0` for 'airplane', `1` for 'automobile', `2` for 'bird', etc. So, `train_labels[0]` might be `[6]` (for 'frog').
  - The model's final **Dense** layer has 10 outputs (one for each class), and the **softmax** activation converts these into a probability distribution (e.g., `[0.01, 0.05, 0.8, 0.02, ...]`).
  - **`sparse_categorical_crossentropy`** internally converts the integer label into a "one-hot" representation (e.g., if the true label is `2`, it becomes `[0, 0, 1, 0, 0, 0, 0, 0, 0, 0]`) before comparing it with the model's predicted probability distribution. This is done efficiently behind the scenes.
- **`categorical_crossentropy` (for contrast)**: This is used when true labels are **already in one-hot encoded format**.
  - **Example**: If your labels were `[0, 0, 1, 0, 0, 0, 0, 0, 0, 0]` for 'bird' instead of just `2`.
  - If your labels are already one-hot encoded, you would use **`categorical_crossentropy`**

# CNN implementation

**Trains the Model:** The model is trained on the training data for a specified number of epochs, with validation performed on the test data.

Epochs - how many times the model will go through the entire training dataset.

Validation data - provides data to evaluate the model's performance on unseen data during training, which helps detect overfitting.

**Evaluates the Model:** After training, the model's performance (loss and accuracy) is evaluated on the unseen test dataset.

**Visualizes Training History:** Plots are generated to show how accuracy and loss changed over epochs for both training and validation sets, which helps in identifying overfitting or underfitting.

**Makes and Displays Predictions:** It picks a few random images from the test set, makes predictions, and displays the image along with its true and predicted labels (colored green for correct, red for incorrect).

# CNN Implementation

<https://colab.research.google.com/drive/1SiycnwdU-gH3sWpAxF4npPDHVLuuJwrf?usp=sharing>